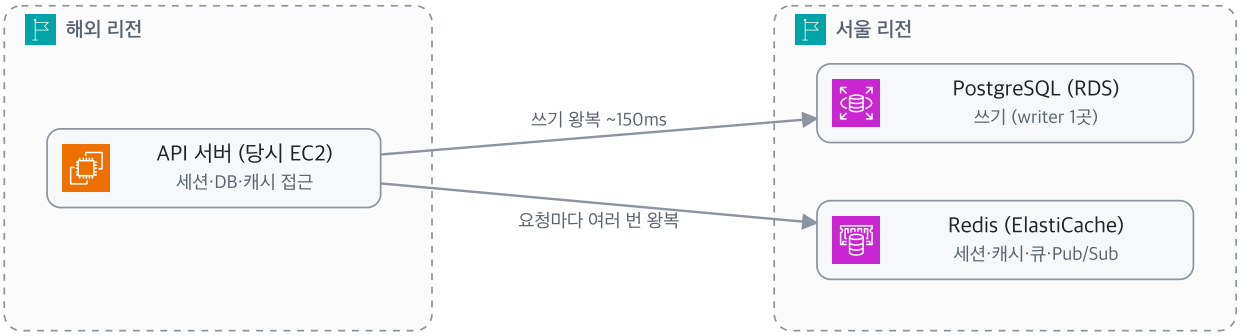
경력기술서

플리토 · 실시간 AI 통번역

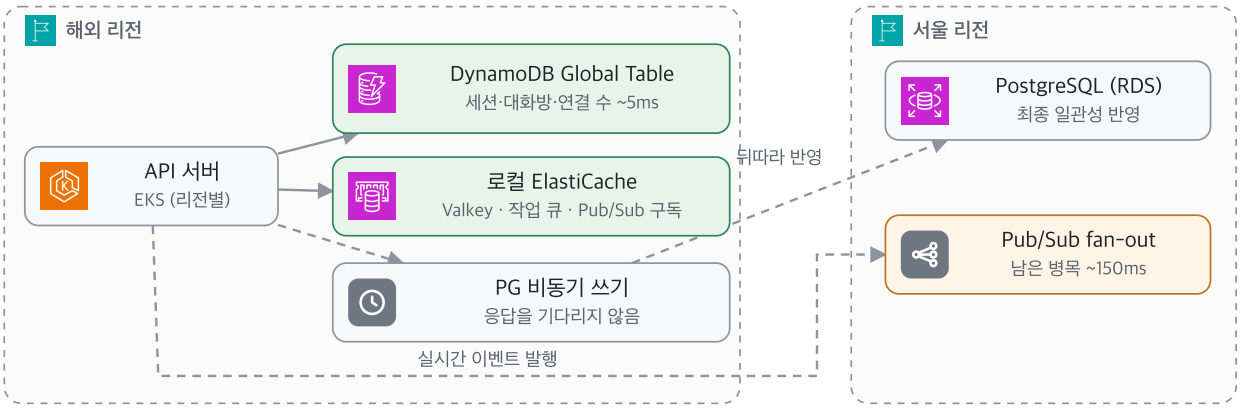
글로벌 리전 레이턴시 개선

2025.11 – 2026.05 | 설계 · 구현 주도 | DynamoDB · Redis · PostgreSQL | 재현 실험 코드 →

Before — 모든 쓰기가 서울 리전으로 왕복



After — 리전 로컬 쓰기 + 비동기 동기화



문제 해외 리전에서 쓰면 국내보다 4~5배 느리다는 피드백. 유력한 원인은 서울 리전에 묶인 DB·Redis 쓰기 왕복(1회 약 150ms)이 요청마다 여러 번 발생하는 구조.

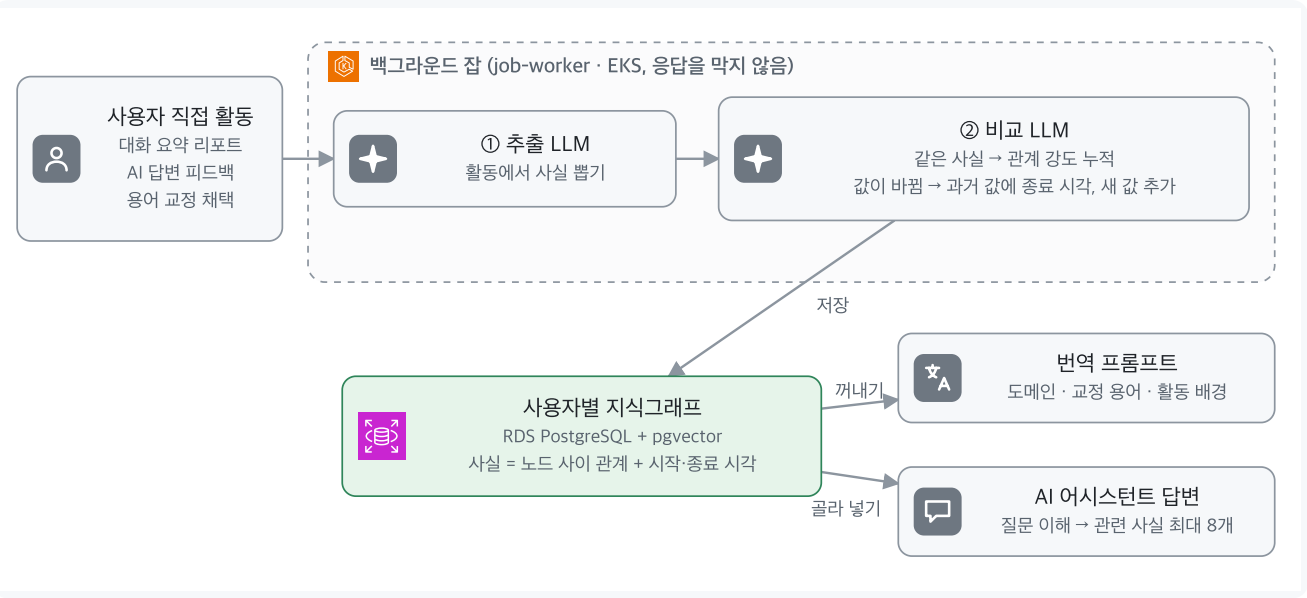
판단 Aurora Global Database·Aurora DSQL 검토 — 쓰기는 여전히 한 리전이거나 대륙 간 미지원이고 비용도 더 높아 보류. 실시간성이 강한 데이터만 DynamoDB와 Redis를 활용한 리전 로컬 쓰기로 옮기기로 결정.

해결 대화방·세션·참가자 연결 수를 DynamoDB Global Tables로, 작업 큐를 리전별 Redis로 옮기고, 실시간 메시지는 모든 리전에 발행해 각 리전에서 구독. PostgreSQL은 응답을 기다리지 않고 뒤따라 쓰게 하고(DynamoDB 실패 시 PostgreSQL로 처리), 설정값 하나로 DynamoDB를 끄면 PostgreSQL에 바로 쓰는 기존 구조로 돌아가게 함.

결과 Close API **1,673 → 327ms (-80%)**, Create API **2,630 → 90ms (-97%)** — 해외 상황을 재현한 개발 환경에서 15회 측정. 남은 병목은 Pub/Sub fan-out.

지식그래프 기반 초개인화

2026.01 – 2026.10 | 백엔드 설계 · 구현 · 상용 반영 | PostgreSQL · pgvector · LLM



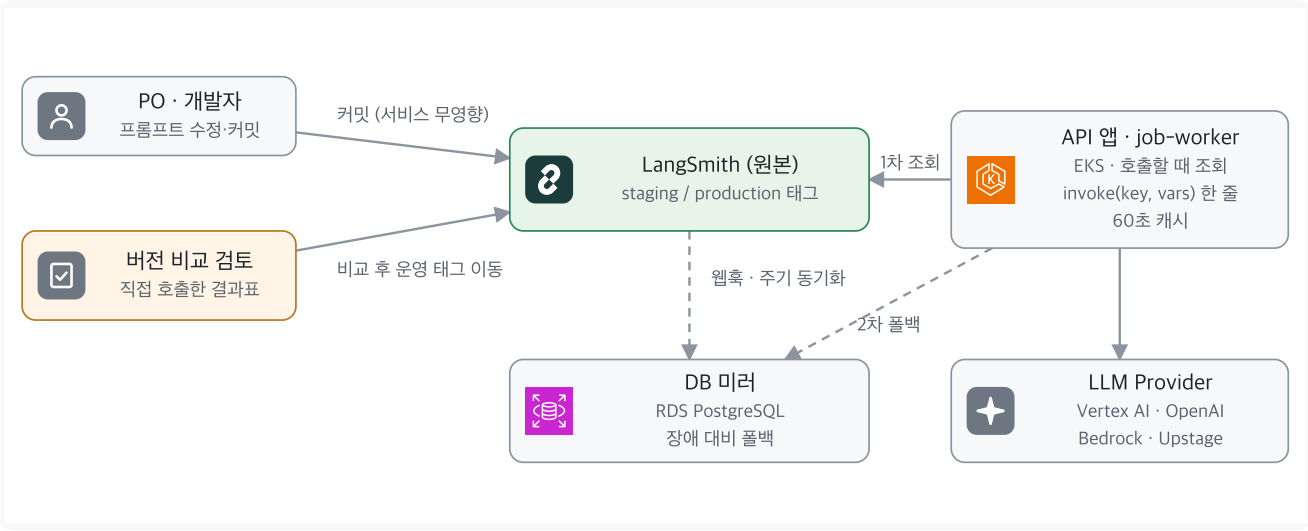
문제 사용자가 여러 번 고친 용어, 직접 준 지시, 자신의 분야와 대화 맥락을 번역 · AI 답변이 기억하지 못해, 쓸수록 나에게 맞춰지는 개인화를 체감할 수 없었음.

해결 사용자의 행동(용어 수정 · AI에 준 지시 등)과 정보를 LLM이 뽑아, 무엇과 무엇이 어떤 관계인지를 시간 정보와 함께 사용자별 지식그래프로 저장 · 업데이트하고, 정보가 바뀌면 최신 정보를 우선으로 번역 · AI 답변에 활용. 쌓는 작업은 백그라운드로 처리.

개선 상용 데이터를 보니 노트 발표자의 이야기가 사용자 사실로 섞이는 문제가 있어 "사용자 중심" 관계 구조로 재설계. 고정 기준선 게이트로 회귀 없이 채택한 뒤 상용 사용자 이관 완료.

LLM 프롬프트 관리 체계

2026.06 설계 · 구현 LangSmith



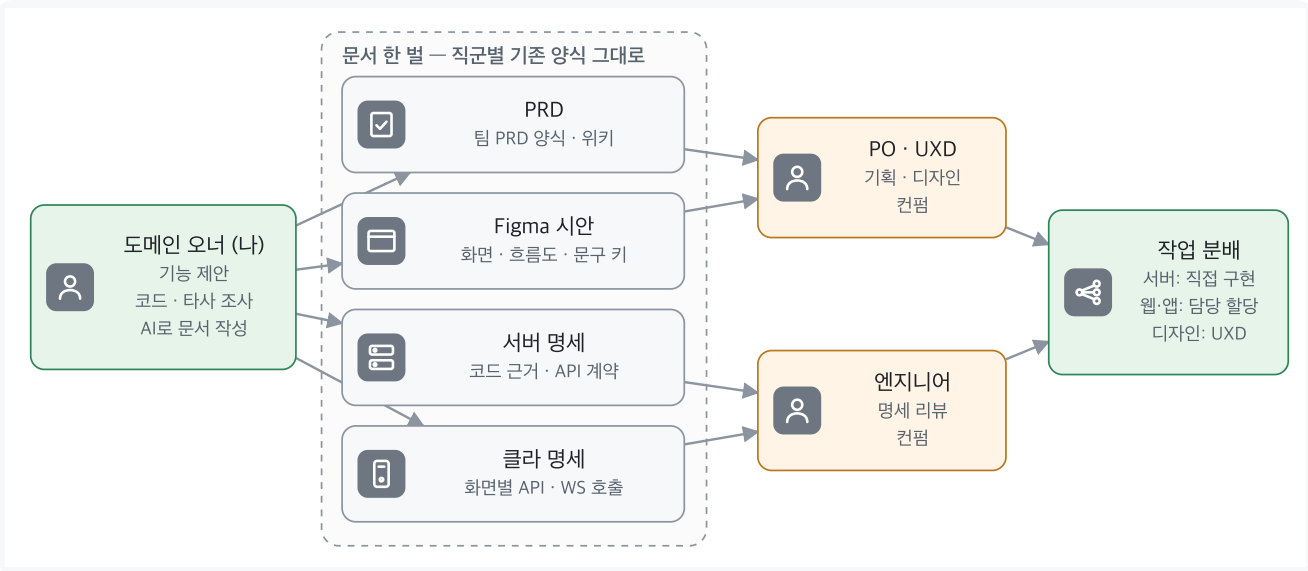
문제 프롬프트가 코드 안 문자열이라 문장 하나를 고쳐도 배포가 필요했고, 프롬프트 버전 관리 자체가 없어 고칠 때마다 생긴 버전들 사이의 차이와 품질을 비교할 방법이 없었음.

해결 LangSmith를 원본으로 두어 프롬프트를 커밋 단위로 버전 관리하고, 서버는 호출할 때 직접 조회(60초 캐시, 장애 시 DB 미러 폴백). 개발/운영은 태그로 분리하고, 새 버전은 개발 태그로 먼저 써 본 뒤 에이전트를 활용하여 프롬프트를 직접 호출한 테스트 결과표로 이전 버전과 비교하고 운영 태그를 옮기는 순서를 팀 릴리스 절차로 정함.

결과 코드 배포 없이 프롬프트를 고쳐 약 1분 안에 반영 — PO·PM용 사용 가이드를 함께 제공.

제품 도메인 오너 — 기획부터 개발 분배까지 문서 한 벌로

2026.10 – 진행 중 | PRD · Figma 시안 · 서버/클라 명세 | 외부 AI 에이전트 연결(MCP) 외



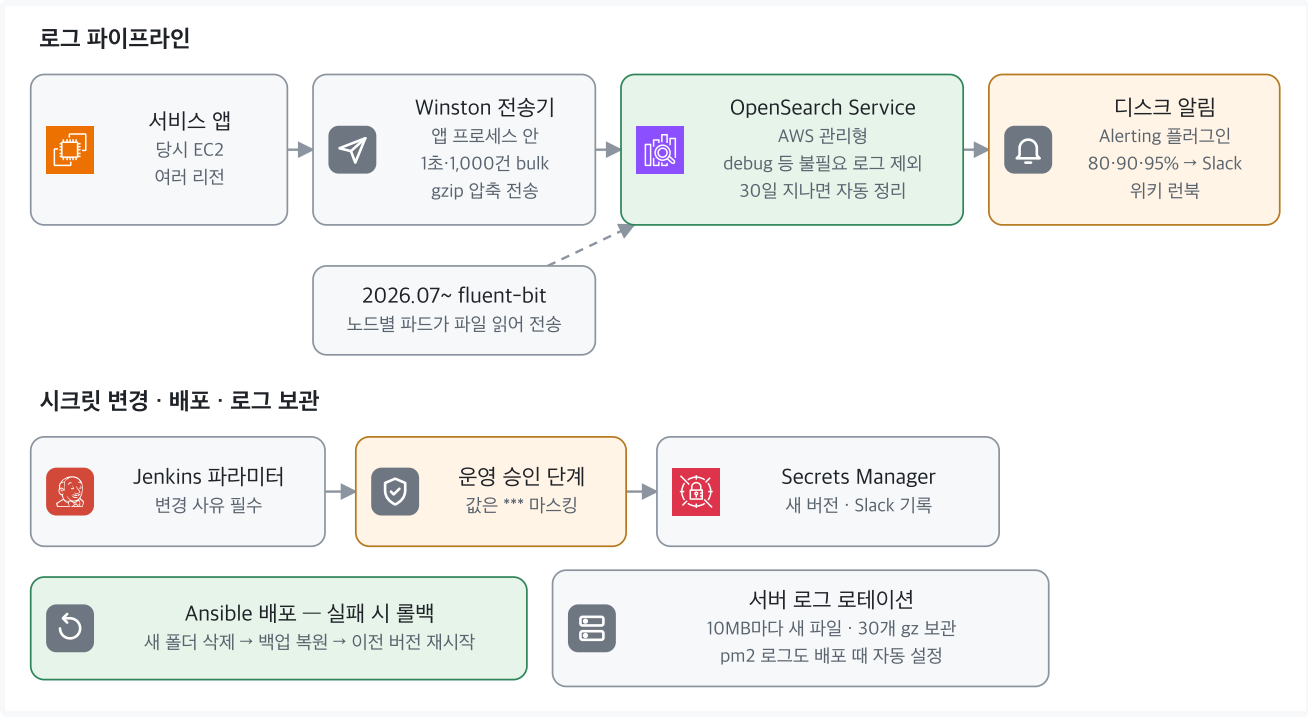
문제 AI 전환(AX)을 위한 논의는 많았지만, 기존 작업 방식의 관성 때문에 실제 변화로 이어지지 않음.

해결 제품 안에서 도메인 하나를 직접 맡아, 각 직군이 이미 쓰던 문서를 참고해 PRD(팀 PRD 양식) · Figma 시안(기존 화면 기준 화면 · 흐름도 · 문구 키) · 서버 명세(코드 근거) · 클라 명세(시안 화면별 API · WebSocket 호출)를 AI 도구와 함께 한 벌로 작성. 내 주 분야가 아닌 기획 · 디자인은 PO · UXD 담당자의 컨펌으로 전문성을 보완하고, 확정된 문서를 기준으로 서버는 직접 구현하고 웹 · 앱은 담당 엔지니어에게 나눠 한 도메인의 전체 작업 흐름을 관리.

진행 첫 적용인 외부 AI 에이전트 연결(MCP)은 타사 13곳의 연결 방식을 비교해 PRD · 시안 · 서버 명세까지 작성하고 검토 중. 같은 절차를 정리해 우선순위가 높은 백로그 도메인 작업에 적용 중.

로그 · 배포 운영 안정화

2025.10 – 2026.05 | 구축 · 장애 복구 | OpenSearch · Jenkins · Ansible



로그

문제 로그가 리전별 서버 디스크에 파일로만 저장돼, 문제가 생기면 서버마다 접속해 찾아야 했음.

해결 동료와 함께 Winston 로거에 OpenSearch 전송기를 직접 구현 — 앱 프로세스 안에서 로그를 모아 1초 또는 1,000건마다 bulk API로 gzip 압축 전송하고, 전송이 실패해도 서버 동작에는 영향이 없게 함(파일 로그는 그대로 유지). 2026.07 k8s 전환 후에는 앱은 파일만 쓰고, 노드마다 뜨는 fluent-bit 파드(DaemonSet)가 그 파일을 읽어 보내는 방식으로 바꿈.

장애 이후 OpenSearch 디스크가 가득 차 로그가 쌓이지 않는 장애 — debug 같은 불필요한 로그는 보내지 않도록 막고, 30일이 지난 로그는 자동으로 정리되도록 인덱스 설정을 바꿈.

감지 AWS OpenSearch에 기본 포함된 Alerting 플러그인으로 5분마다 디스크를 확인해 80 · 90 · 95%를 넘으면 팀 Slack 채널로 알림. 알림에는 사용량을 확인하고 옛 인덱스를 정리하는 가이드 문서(위키) 링크를 첨부.

배포

문제

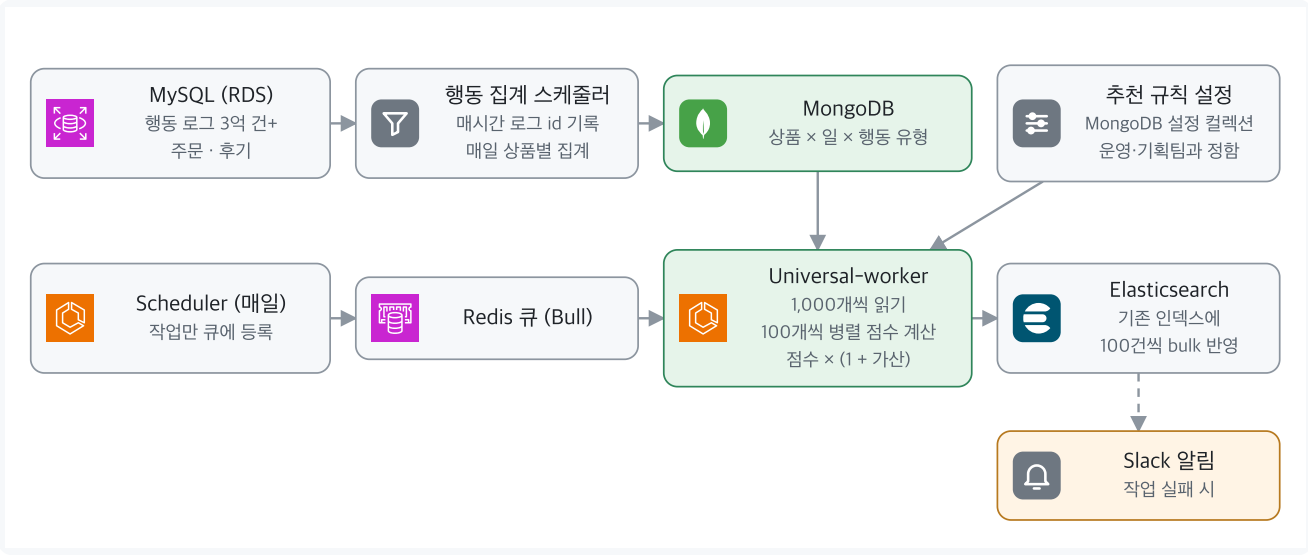
- **시크릿** 누가 어떤 값으로 언제 바꿨는지 기록되지 않아, 시크릿 변경이 원인인 이슈를 추적해 대응하는 일이 생김
- **로그** 상용 웹 서버 CPU 포화 장애 때 서버 디스크의 로그 파일이 정리되지 않고 쌓여 있었음
- **롤백** 배포 스크립트에 실패하면 이전 버전으로 되돌리는 구조가 없었음

해결

- **시크릿** Jenkins 파이프라인으로 일원화 — 변경 사유 필수 · 값은 *** 마스킹해 Slack에 기록 · 상용은 승인 후 실행
- **로그** 로그 로테이션으로 10MB마다 새 파일을 만들고 30개까지만 압축 보관해, 오래된 로그는 자동으로 지워지게 함
- **롤백** Ansible 배포에 롤백 단계를 추가해, 실패하면 백업을 복원하고 이전 버전으로 다시 실행

추천 상품 스케줄러 재구축

설계 · 구현 MySQL · MongoDB · Elasticsearch · Bull



방식 개인화 모델이 아니라, 브랜딩·운영팀과 기획자가 통계 데이터를 보고 정한 규칙(행동별 가중치)으로 상품 점수를 매겨 추천 순서를 내려주는 룰 기반 추천. 트렌드순 · 인기순은 운영 담당자가 정한 유효 범위의 데이터(기간 · 행동 유형) 기준으로 점수를 가산해 순서를 매김.

문제 상품마다 3억 건 이상의 행동 로그에서 2주치 클릭·좋아요·장바구니 수를 세는 서브쿼리를 붙이고 주문·리뷰까지 함께 조회해 실행에 60분 이상 걸림. 가중치(예: 클릭 5 · 좋아요 10 · 장바구니 20)도 쿼리에 고정돼 있었지만, 실행 부하가 커서 3년 넘게 손대지 못함.

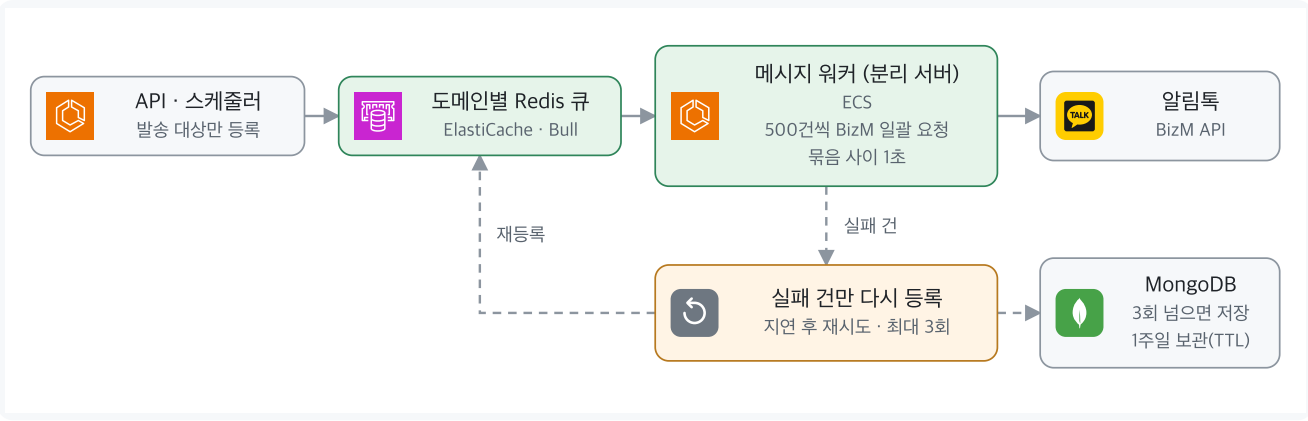
해결 3억 건 이상의 원본 로그를 매번 조회하는 대신, 매시간 그 시각의 로그 id를 MongoDB에 기록해 두고 매일 이 id 범위로 상품별 · 행동 유형별 하루 집계를 MongoDB에 저장. 점수 계산은 원본 로그가 아닌 이 집계를 읽어 상품을 1,000개씩 읽고 100개씩 병렬로 계산하도록 다시 작성하고, 스케줄러는 작업만 큐에 넣고 계산은 별도 워커(Universal-worker)가 맡도록 분리. 가중치 · 기간 · 정렬 조건은 쿼리에서 빼 MongoDB 설정 컬렉션으로 옮겨, 코드 배포 없이 내부 관리자가 값을 조율하며 추천 결과를 볼 수 있게 함. 작업이 실패하면 Slack으로 알림.

판단 전체 상품을 새 인덱스로 만들어 통째로 바꾸는 방식(별칭 교체 · 블루그린 인덱스)은 검색 쿼리가 인덱스 이름을 직접 지정하고 있어 모든 검색 쿼리를 함께 고쳐야 하고, 데이터 교체와 배포 사이에 검색이 잠깐 끊길 위험이 있었음 → 인덱스는 그대로 두고 상품 문서의 추천 순번(트렌드 · 인기)만 100건씩 bulk로 차례로 갱신(충돌 시 재시도).

결과 스케줄러 실행 시간 **60분 → 15분**.

알림 메시지 워커

설계 · 구현 Bull(Redis) · ECS · MongoDB



문제 쿠폰 만료 안내 같은 알림톡을 약 100만 명에게 주기적으로 보내는 작업이 API 서버에 큰 부담을 줌. 발송에 실패한 메시지를 다시 보내거나 기록하는 체계도 없었음.

해결 발송을 별도 워커 서버(ECS)로 분리. API · 어드민 · 스케줄러는 발송 대상만 도메인별 Redis 큐(Bull — 쿠폰 · 문의 답변 등)에 넣고, 워커가 500건씩 묶어 BizM에 일괄 요청하며 묶음 사이에 1초를 둠. 응답에서 실패한 건만 모아 시도 횟수를 붙여 지연 후 큐에 다시 넣고(성공한 건은 다시 보내지 않음), 3회를 넘으면 로그 워커 큐를 거쳐 MongoDB에 실패 이력을 1주일 보관(TTL).

판단

- **큐 단위** 처음엔 채널(카카오톡 · SMS) 기준 큐로 설계했다가, 무엇으로 보낼지는 도메인 잡이 정하도록 도메인 기준으로 다시 설계 — 새 알림은 잡 하나로 추가
- **속도** 500건 · 1초 간격은 BizM 기술팀에 문의한 권장 처리량(초당 200~250건)과 우리 작업 시간을 함께 보고 정함
- **중복** 수신 번호 기준 중복 제거는 100만 건을 메모리에서 거르면 속도 · 메모리 부담이 커서 DB 조회 단계로 옮김
- **인프라** 다른 워커에서 이미 쓰던 Bull을 그대로 써서 새 인프라 없이 분리

Elasticsearch 스냅샷 마이그레이션 자동화

설계 · 구현 | Step Functions · Lambda · EventBridge · S3



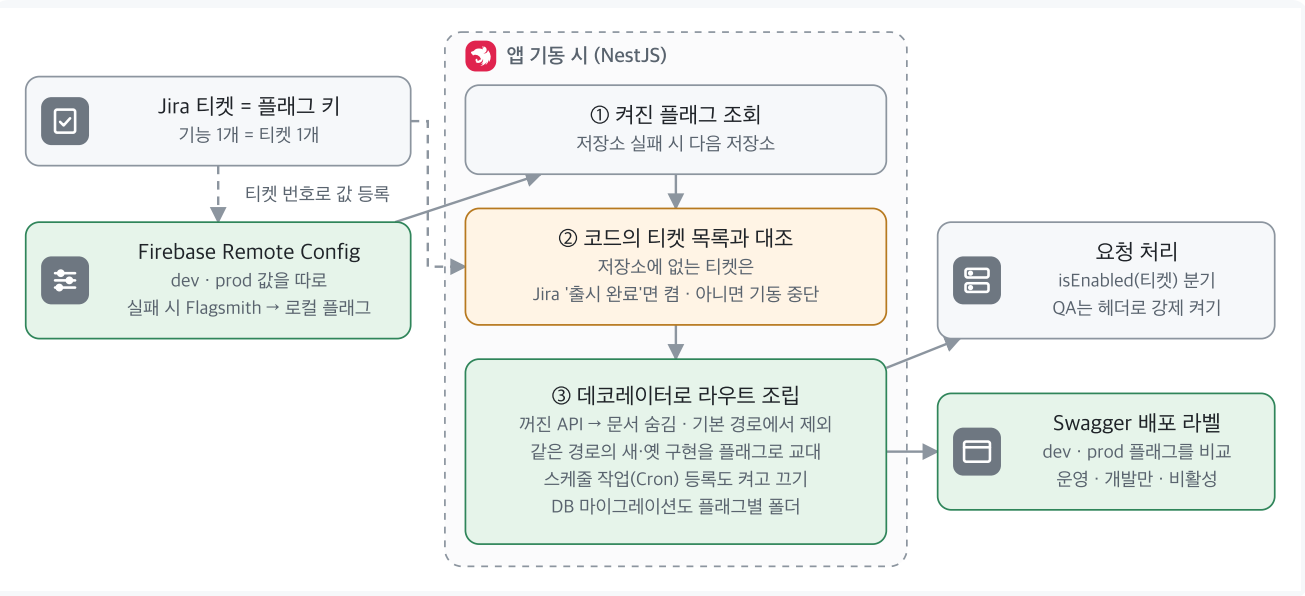
문제 스테이징 환경을 새로 만들면서 개발 환경의 검색 데이터를 옮기고 주기적으로 맞춰야 했는데, 스냅샷 생성 → 완료 확인 → 복원을 사람이 순서대로 해야 했고 중간에 실패해도 알아채기 어려움.

해결 EC2에 설치된 Elasticsearch에 repository-s3 플러그인과 S3 접근 IAM 역할을 설정해 S3를 스냅샷 저장소로 연결(플러그인 적용에 재시작이 필요해, 운영 클러스터는 사내에 미리 알리고 마스터 노드를 가장 마지막에 재시작). 개발 → S3 스냅샷 생성 → 완료될 때까지 상태 확인(실패면 중단) → 스테이징 복원 → 클러스터·노드 상태 확인을 Step Functions 단계로 묶고, 각 단계는 VPC 안에서 ES를 호출하는 Node.js Lambda로 작성. EventBridge로 주기 실행.

판단 스냅샷은 데이터 크기에 따라 오래 걸려 Lambda 한 번(최대 15분) 안에서 기다리기 어려움 → 요청과 확인을 짧은 단계로 나누고, 순서·대기·실패 분기는 Step Functions가 맡게 함.

Feature Flag 기반 Trunk-Based 개발

팀 도입 · Swagger 배포 라벨 구현 | Firebase Remote Config · NestJS · GitHub Actions



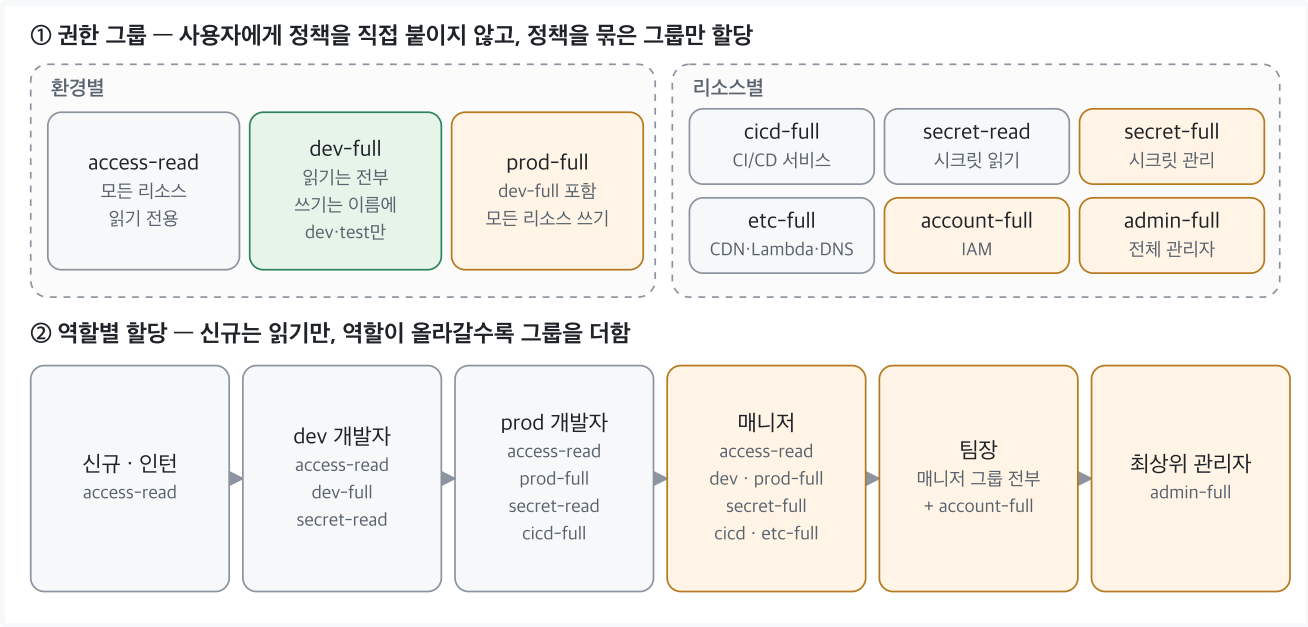
문제 dev · release 브랜치를 따로 관리해 공통 코드 작업이 자주 충돌하고, 운영 배포가 늦어지면 개발 환경과 내용이 어긋남. 공통 모듈을 고쳐도 운영 배포 전까지는 다른 작업에서 쓰기 어려움.

방식 팀에서 master 하나로 개발하고 기능은 플래그로 켜고 끄는 방식을 도입. 플래그 키는 Jira 티켓 번호, 값은 Firebase Remote Config에 개발 · 운영을 따로 둠(조회 실패 시 Flagsmith → 로컬 플래그). 앱이 뜰 때 켜진 플래그를 읽어 라우트를 조립 — 꺼진 API는 문서에서 숨기고 기본 경로에서 빼며, 같은 경로의 새 · 옛 구현을 플래그로 교대하고, 스케줄 작업과 DB 마이그레이션도 플래그별로 켬. 코드에는 있는데 저장소에 없는 티켓은 Jira에서 '출시 완료'인 것만 켜고 아니면 기동을 멈춰 설정 누락을 막음. master에 병합하면 GitHub Actions가 dev · release PR을 자동 생성하고, 문제가 생기면 버전을 되돌리는 대신 플래그로 기능을 닫음(Firebase 장애 시에는 켤 티켓을 코드에 직접 적어 배포). 출시 7일이 지난 티켓은 알림을 받아 플래그와 옛 코드를 지움.

내 구현 동료가 만든 플래그 모듈 위에, 개발 · 운영 플래그를 비교해 Swagger에 API별 배포 상태 라벨(운영 · 개발만 · 비활성)을 자동으로 붙이는 기능 — 클라이언트 개발자가 오래 요청하던 "이 API가 어디까지 배포됐나"를 문서에서 바로 확인.

AWS 권한 정책

설계 · 문서화 | IAM 그룹 9개 · 역할 6단계



문제 백엔드팀이 모든 권한을 가진 단일 그룹을 함께 써서, 신규 입사자도 운영 리소스를 바꾸거나 지울 수 있는 상태였음.

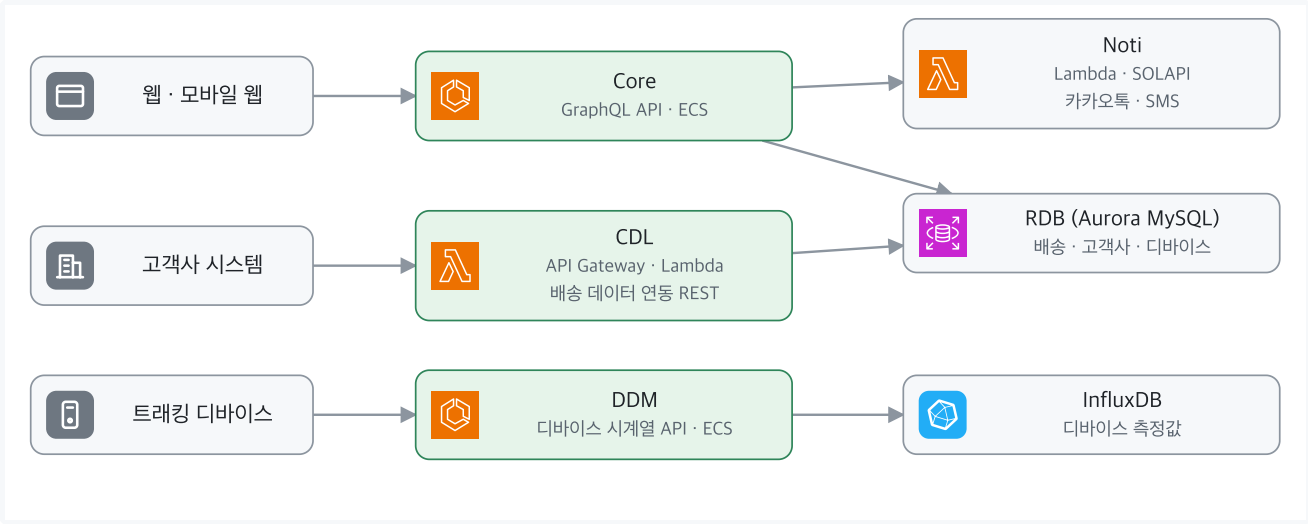
해결

- **그룹** 사용자에게 정책을 직접 붙이지 않고, 정책을 묶은 권한 그룹 9개만 할당 — 환경별(전체 읽기 · dev · prod)과 리소스별(CI/CD · 시크릿 읽기/관리 · CloudFront · Lambda · Route53 · IAM · 관리자)로 나눔
- **제한** dev 그룹은 읽기는 전부 허용하되 생성 · 수정 · 삭제는 이름 · 태그에 dev · test가 붙은 리소스로만 제한해 운영 리소스를 실수로 건드릴 수 없게 함. prod 그룹은 dev 그룹을 포함
- **할당** 신규 · 인턴은 읽기 전용만 받고, dev 개발자 → prod 개발자 → 매니저 → 팀장으로 갈수록 그룹을 더함. 시크릿 관리 · IAM · 관리자 권한은 매니저급 이상만
- **절차** 추가 권한은 매니저급 이상에게 요청해 받고, 주기적으로 권한을 검토해 불필요한 권한을 제거. 원칙 · 그룹 정의 · 리소스별 정책 · 부여 절차를 문서로 정리

윌로그 · 물류 배송 모니터링

산업군 통합 관제 플랫폼 — 기능 기준 서버 분리

2022.12 – 2023.09 · 백엔드 설계 · 구현 · NestJS · GraphQL · ECS · Lambda · InfluxDB



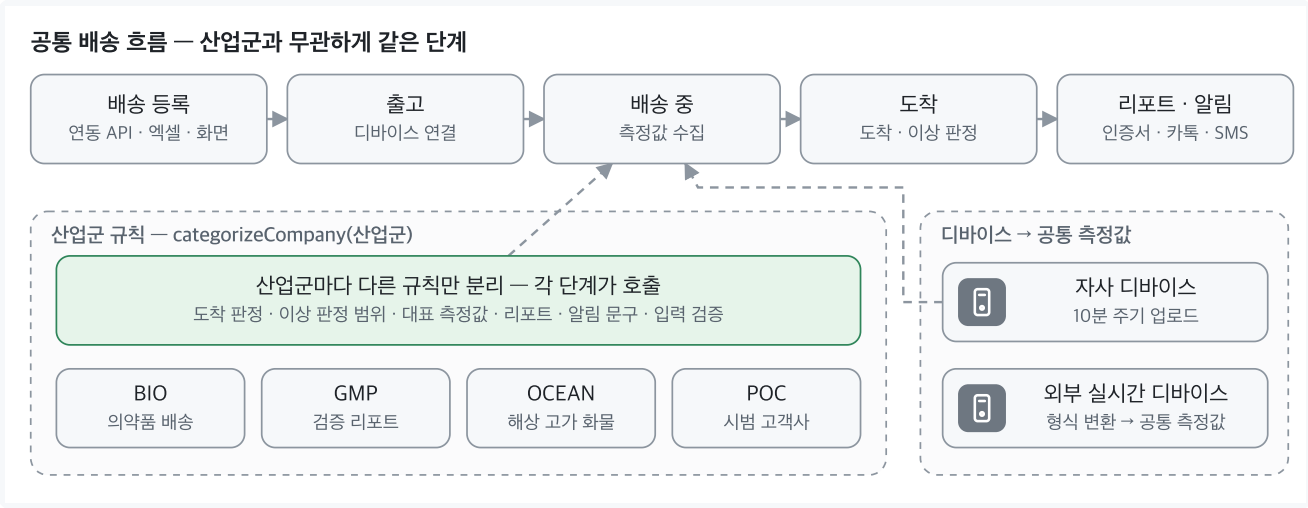
문제 의약품 배송 모니터링에서 시작해 산업군마다 서비스를 따로 만들다 보니 관리할 서버와 엔드포인트가 계속 늘어날 상황. 레거시에서는 고객사 연동 API 트래픽이 몰리면 서비스 전체가 영향을 받은 적이 있음.

해결 배송 도메인을 하나로 합친 플랫폼을 설계하는 데 참여해 백엔드를 맡음. 부하 성격이 다른 기능끼리 서버를 나눔 — 화면용 GraphQL API(Core), 디바이스 시계열(DDM), 고객사 연동 REST(CDL), 카카오톡 · SMS 알림(Noti).

판단 상시 트래픽을 받는 Core · DDM은 ECS, 요청이 몰렸다 사라지는 연동 · 알림은 Lambda로 둬 연동 폭주가 화면 API에 번지지 않게 하고, 쓰는 만큼만 비용을 냄.

공통 배송 흐름 + 산업군 규칙

의약품 · GMP 검증 · 해상 고가 화물 | 디바이스 호환



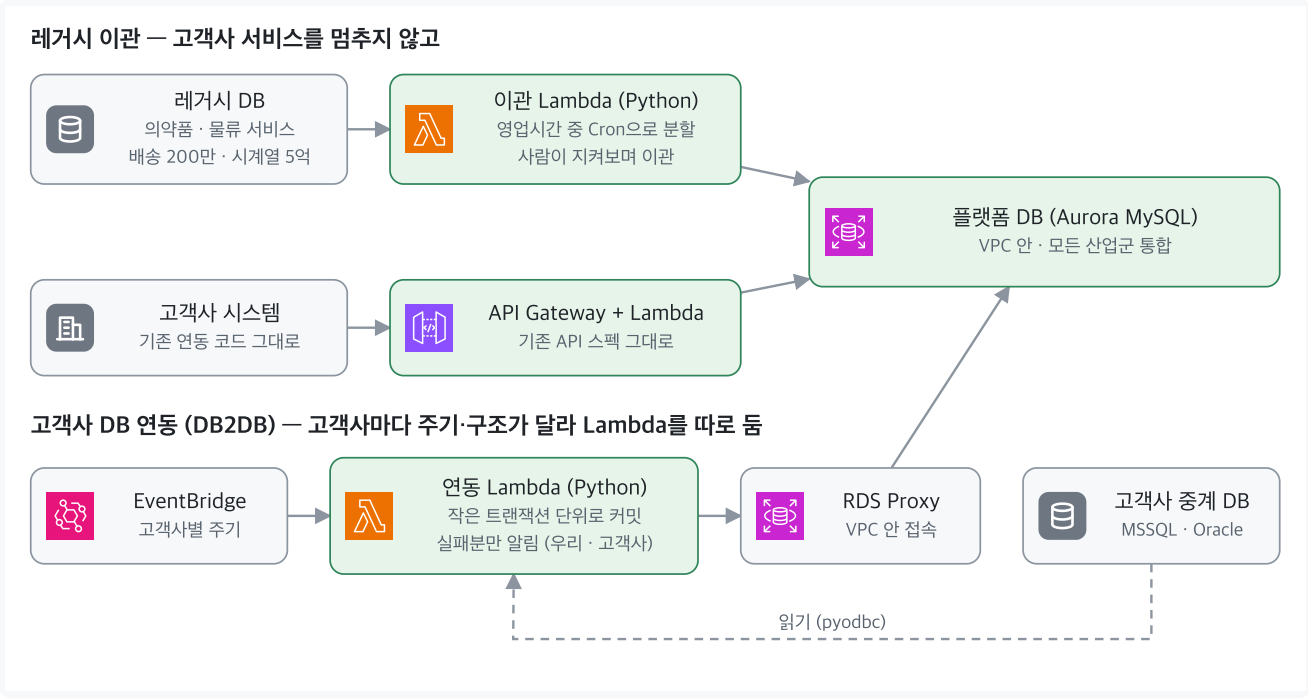
문제 의약품 · GMP 검증 · 해상 고가 화물은 도착을 판정하는 기준, 이상으로 보는 온도 · 습도 범위, 리포트 양식, 알림 문구가 서로 다름. 산업군마다 서비스를 새로 만들면 같은 배송 로직이 계속 복제됨.

해결 등록 → 출고 → 배송 중 → 도착 → 리포트 · 알림 흐름은 하나로 두고, 산업군마다 다른 규칙(도착 판정 · 이상 판정 범위 · 대표 측정값 · 리포트 · 알림 문구 · 입력 검증)만 산업군 값 하나로 고르는 규칙 묶음으로 분리. 자사 디바이스와 외부 실시간 디바이스도 공통 측정값 형식으로 바꿔 같은 흐름에 넣음. 예: 도착은 전체 도착 · 부분 도착 · 임의 도착 시간으로, 반품은 부분 · 전체 반품으로 나누고, 이상 판정 범위는 고객사 → 배송그룹 → 배송 순으로 상위 값을 기본값으로 물려받게 함.

결과 MVP 이후 배차 관제, 제약사 GMP 검증 플랫폼, 해상 고가 화물 플랫폼을 같은 코드베이스에 차례로 출시 (2023.03 – 2023.08).

레거시 무중단 이관 · 고객사 DB 연동

2023 Python Lambda · API Gateway · EventBridge · RDS Proxy · Aurora

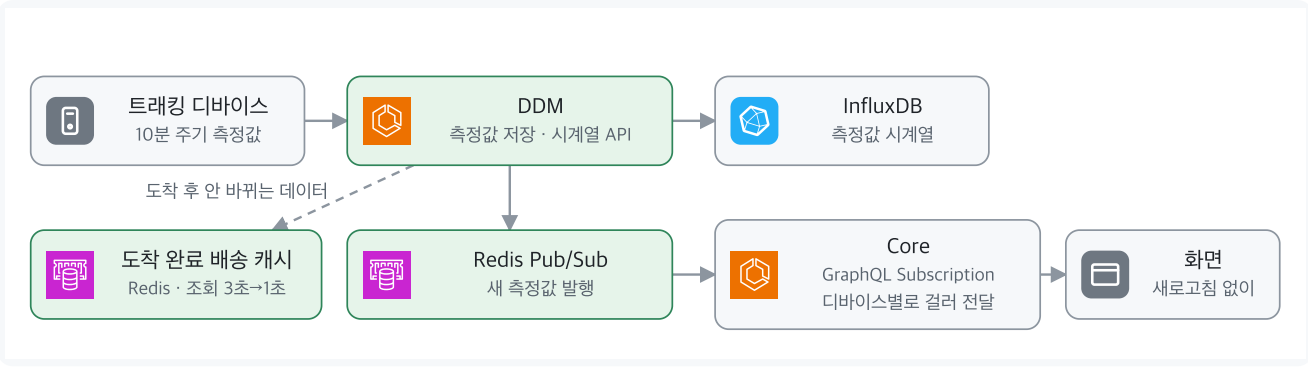


이관 두 레거시 서비스(의약품 · 물류)의 테이블과 기능 차이를 플랫폼 기준 대조표로 정리한 뒤(예: 의약품 서비스의 주문 단위 → 플랫폼 배송그룹), 운영 중인 고객사 서비스를 멈추지 않도록 테크 리드와 협의해 데이터는 Python Lambda로 영업시간에 Cron으로 나눠 사람이 지켜보며 옮기고, 고객사가 쓰던 API는 요청 · 응답 형식을 그대로 맞춰 API Gateway + Lambda로 다시 구축 — 고객사는 코드를 바꾸지 않고 전환. 배송 약 **200만 건** · 디바이스 시계열 약 **5억 건** 이관.

연동 고객사가 중계 DB(MSSQL · Oracle 등)를 두는 방식은 고객사마다 주기 · 구조가 달라 고객사별 Lambda와 EventBridge 스케줄로 구성. 처음엔 Node.js였지만 DB 드라이버가 다양하고 계약 전 POC에서 가장 먼저 붙여야 하는 작업이라 Python으로 통일. Lambda가 실행될 때마다 DB 연결을 새로 맺지 않도록 VPC 안 Aurora에는 RDS Proxy 연결 풀로 접속(다른 서비스와 함께 쓰는 DB라 풀의 30%까지만)하고, 서버리스 전용 DB 사용자를 따로 만들어 Secrets Manager 자동 암호 교체와 IAM 인증을 적용. 쓰기는 작은 트랜잭션 단위로 나눠 성공분은 커밋하고 실패분만 우리 쪽과 고객사에 알림.

실시간 측정값 · 조회 성능

InfluxDB · Redis Pub/Sub · GraphQL Subscription



시계열	배송 테이블에 콤마로 이어 붙인 문자열로 저장하던 측정값(온도 · 습도 · 위치 · 측정 시간)을 InfluxDB로 분리. 기존 TypeORM처럼 쓰는 조회 모듈을 만들어 서비스 코드 변경을 최소화.
실시간	DDM이 새 측정값을 Redis Pub/Sub으로 발행하고, Core의 GraphQL Subscription이 구독할 때 받은 디바이스 목록으로 걸러 화면에 새로고침 없이 전달. WebSocket은 쿠키에 토큰을 담을 수 없어, 연결할 때 받은 토큰을 요청 context에 넣어 기존 인증 가드로 확인.
캐시	배송 기간이 길어지면서 시계열 조회가 느려짐 → 도착한 배송은 데이터가 더 바뀌지 않는다는 점을 이용해 Redis에 캐시, 조회 약 **3초 → 1초**.

문제 해결 · 협업

고객사 연동 API 지연

고객사가 10분마다 1만 건 이상 보내는 배송 데이터를 저장 전에 1건씩 라벨 코드 포함 검색(LIKE)으로 중복 검사해 slow query 발생(CloudWatch에서 확인) → 라벨 코드를 100개씩 묶어 한 번에 조회해 쿼리 수를 1/100로 줄이고(1만 건 기준 1만 회 → 100회), 조회 기간은 1개월에서 4일로 좁힘. 고객사 · 배송 정보는 메모리(Record)에 담아 다시 조회하지 않음. 실행 테스트 평균 **20초 → 5초**, 이후 연동 API를 Lambda로 분리.

사내 관리자 서비스(RMS)

NestJS 백엔드를 처음 맡아 설계 · 구현(2022.01 – 03). 디바이스 재고 · 고객사 계정 · 미팅 기록을 한곳에서 관리하고, 관리자가 내부 키로 인증하면 JWT를 발급해 고객사 계정 정보 없이 운영 서비스에 바로 들어가 CS를 처리하게 함.

VOC time

니즈가 정리되지 않은 고객사에 물류 배송 과정을 바탕으로 한 MVP를 먼저 제공하고, 안정화될 때까지 한 달에 한 번 현장 실무자를 만나 기능을 맞춤 개발.